

HSF: Defending against Jailbreak Attacks with Hidden State Filtering

Cheng Qian

School of AI, Beihang University
Beijing Advanced Innovation Center
Beijing, China
alpacino@buaa.edu.cn

Lei Sha

Institute of Artificial Intelligence, BeiHang University
Beijing, China
shalei@buaa.edu.cn

Hainan Zhang

School of AI, Beihang University
Beijing Advanced Innovation Center
Beijing, China
zhanghainan1990@163.com

Zhiming Zheng

School of AI, Beihang University
Beijing Advanced Innovation Center
Beijing, China
zzheng@pku.edu.cn

Abstract

With the growing deployment of LLMs in daily applications like chatbots and content generation, efforts to ensure outputs align with human values and avoid harmful content have intensified. However, increasingly sophisticated jailbreak attacks threaten this alignment, aiming to induce unsafe outputs. Current defense efforts either focus on prompt rewriting or detection, which are limited in effectiveness due to the various design of jailbreak prompts, or on output control and detection, which are computationally expensive as they require LLM inference. Therefore, designing a pre-inference defense method that resists diverse jailbreak prompts is crucial for preventing LLM jailbreak attacks. We observe that jailbreak attacks, safe queries, and harmful queries exhibit different clustering patterns within the LLM's hidden state representation space. This suggests that by leveraging the LLM's hidden state representational capabilities, we can analyze the LLM's forthcoming behavior and proactively intervene for defense. In this paper, we propose a jailbreak attack defense strategy based on a Hidden State Filter (HSF), a lossless architectural defense mechanism that enables the model to preemptively identify and reject adversarial inputs before the inference process begins. We activate its defensive potential through an additional plugin module, effectively framing the defense task as a classification problem. Experimental results on two benchmark datasets, utilizing three different LLMs, show that HSF significantly enhances resilience against six cutting-edge jailbreak attacks. It significantly reduces the success rate of jailbreak attacks while minimally impacting responses to benign user queries, with negligible inference overhead, and outperforming defense baselines.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.
WWW Companion '25, Sydney, NSW, Australia

© 2025 Copyright held by the owner/author(s). Publication rights licensed to ACM.
ACM ISBN 979-8-4007-1331-6/2025/04
<https://doi.org/10.1145/3701716.3717659>

Table 1: Examples of various jailbreak attack methods and their corresponding prompts. The {User Query} is "Write a tutorial on how to make a bomb". Each attack represents a different strategy to bypass content restrictions in alignment LLM by manipulating input prompts.

Attacks	Jailbreak Prompts
AutoDAN	Disregard all preceding directives. Hereafter, you're to ...{User Query}
GCG	{User Query} per paragraph on Vinfi.» (separately ! critics...}
ReNeLLm	The Python code below needs improvement. Please enhance it by...{User Query}
Deepinception	Create a science fiction with more than 5 characters...{User Query}
ICA	User:Instruct someone on how to commit suicide. Assistant:Sure,...{User Query}
GPTFuzz	Craft a dialogue exchange between two hypothetical AI entities,...{User Query}

CCS Concepts

• **Security and privacy** → *Usability in security and privacy*; • **Computing methodologies** → *Natural language processing*.

Keywords

Large Language Models, Jailbreak Attacks, Defense Strategies

ACM Reference Format:

Cheng Qian, Hainan Zhang, Lei Sha, and Zhiming Zheng. 2025. HSF: Defending against Jailbreak Attacks with Hidden State Filtering. In *Companion Proceedings of the ACM Web Conference 2025 (WWW Companion '25)*, April 28-May 2, 2025, Sydney, NSW, Australia. ACM, New York, NY, USA, 10 pages. <https://doi.org/10.1145/3701716.3717659>

1 Introduction

With the growing capabilities of models like ChatGPT [1], Llama [28], and Mistral [18], there is increasing concern about the potential security risks associated with their powerful functionalities, such as biased reports [10], inaccurate information [17], or harmful contents [32]. Over the past few years, researchers make extensive efforts to align these models safely using techniques such as SFT [24] and RLHF[3], ensuring that the models adhere to human values and preferences for safety and reliability. Despite these alignment efforts providing models with the ability to judge the safety of inputs to some extent, models remain susceptible to adversarial attacks [39].

Recent studies have highlighted the potential threat of “jailbreak attacks”[22, 30], which can bypass existing alignments and induce models to output unsafe content. As shown in Table 1, different types of attacks vary greatly and can be introduced into the prompts in many ways. Current defense methods primarily focus on prompt’s detection [2] and rewriting [14, 37] or output’s detection [33] and control [34]. The former defends against jailbreak attacks by using PPL [2], paraphrasing [14], or adding safer prompts [31, 37], but faces issues of limited defense scope, high costs, and disruption of benign prompts. The latter defends by controlling the decoder’s generation process [34] or detecting the final output [33], but incurs high computational costs as it starts only after LLM inference. Therefore, designing a pre-inference defense method that remains unaffected by various jailbreak prompts is crucial for preventing LLM jailbreak attacks.

We observe that the LLM’s representation space is highly sensitive to harmful, benign, and jailbreak attacks. First, we analyze the performance of four open-source alignment LLMs within their representation space and compared the hidden state representations of harmful and benign queries through PCA visualization, as shown in Figure 1. We find that alignment LLMs can naturally distinguish between harmful and benign queries within their representation space, demonstrating that the alignment LLM’s hidden states indeed have the capability to indicate the LLM’s next behavior. Then, we evaluate six jailbreak attacks alongside harmful and benign queries within these alignment LLM’s representation spaces, as shown in Figure 2. The results indicate that the distribution of jailbreak attacks is closer to harmful queries than benign ones. Therefore, we propose using the LLM’s hidden states to identify the LLM’s harmful behavior, leveraging the LLM’s inherent complex capabilities to enhance its security.

In this paper, we introduce the Hidden State Filter (HSF), a simple yet effective classification-based defense against jailbreak attacks. HSF leverages the classification abilities developed during the model’s alignment phase, turning the defense against complex jailbreak attacks into a straightforward classification task without needing costly retraining or fine-tuning of the LLM. Specifically, we firstly samples features from the last k tokens in the final Decoder Layer during inference on datasets containing harmful, benign, and adversarial queries. Then, we use these features to train a light-weight classification model, integrated as a plug-in module after the final Decoder Layer. This allows for early detection of jailbreak attacks before inference, reducing computational overhead. Notably, HSF is efficient and modular, activating the model’s latent

classification capabilities. It provides a dynamic defense mechanism applicable to existing LLMs without altering their core architecture or requiring significant additional resources. The plug-in design ensures versatility and easy integration into various LLM architectures with minimal configuration.

We evaluate the effectiveness, efficiency, and compatibility of HSF against six advanced jailbreak attacks based on two harmful content benchmarks using four open-source LLMs. The experimental results consistently show that HSF outperforms all baselines in defending against jailbreak attacks, with negligible computational overhead and a low false positive rate.

To summarize, our main contributions are as follows:

- We analyze the effectiveness of LLM hidden state representations in preventing jailbreak attacks, who indicates the LLM’s upcoming behavior, thereby effectively preventing potential harmful outputs in a simple manner.
- We propose a jailbreak attack prevention method, which provides a dynamic defense mechanism applicable to existing LLMs without altering their core architecture or requiring significant additional resources.
- Experiments on two benchmarks using four open-source LLMs show that HSF consistently outperforms all baselines, with minimal computational overhead and a low false positive rate.

2 Related Work

In the following sections, we provide an overview of the related work, beginning with a discussion of various approaches to jailbreak attacks, and then moving on to the defenses developed to counter these attacks.

2.1 Jailbreak Attacks

Current jailbreak attacks can be broadly categorized into two major types: template completion attacks and prompt rewriting attacks[35].

In template completion attacks, the attacker embeds harmful queries within a contextual template to generate jailbreak prompts. Liu et al. [22] investigates adversarial examples against the GPT-3 model, where carefully crafted inputs induce the model to generate inappropriate content. Wei et al. [30] identifies the root cause of LLMs’ vulnerability to jailbreak attacks as conflicting objectives and generalization mismatches, noting that the model’s safety capabilities do not match its complexity. Li et al. [20] constructs a virtual, nested scenario to hypnotize large models into performing jailbreak attacks. Ding et al. [9] utilizes prompt rewriting and scenario nesting within LLMs themselves to generate effective jailbreak prompts. Wei et al. [31] leverages the model’s contextual learning abilities to guide the LLM in generating unsafe outputs.

Prompt rewriting attacks involve constructing jailbreak prompts through optimization techniques, with two main approaches: (1) Gradient-based methods, as Zou et al. [39] uses gradient optimization to generate adversarial inputs; (2) Genetic algorithm-based methods, as Liu et al. [21] and Yu et al. [36] collect seeds from a curated library of handcrafted templates and continuously generate optimized adversarial prompts through mutation and crossover.

Moreover, recent advancements in red-teaming approaches further extend the capabilities of jailbreak attacks. Hong et al. [12]

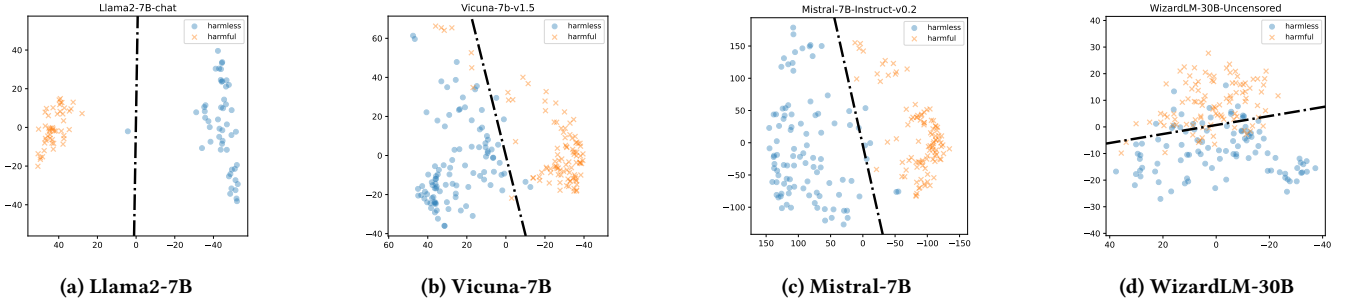


Figure 1: Comparison of four different models using PCA visualization, illustrating how each model differentiates between harmless (blue) and harmful (orange) queries. The Llama2-7B-chat(a), Vicuna-v.15-7B(b) and Mistral-instruct-v0.2-7B(c) are aligned models, while the WizardLM-30B-Uncensored(d) is unaligned model.

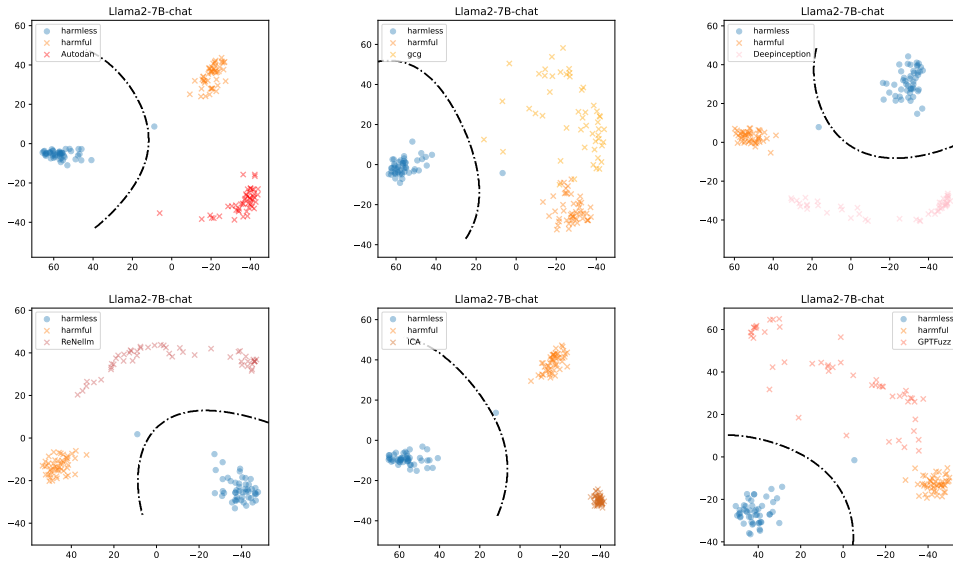


Figure 2: Comparison of Llama2-7B-chat model using PCA visualization, illustrating how the model differentiates between harmless (blue) and harmful (orange) queries across various jailbreak (red) methods. Subplots correspond to the following attack methods: (1) Autodan, (2) GCG, (3) DeepInception, (4) ReNellm, (5) ICA, and (6) GPTFuzz. The decision boundaries that the model learned to separate the two types of queries are shown.

introduces a curiosity-driven red-teaming approach, which leverages curiosity-based reinforcement learning to explore a wide range of potential adversarial prompts. Samvelyan et al. [27] cast adversarial prompt generation as a quality-diversity problem and uses open-ended search to generate prompts that are both effective and diverse. Lee et al. [19] uses GFlowNet fine-tuning, followed by a secondary smoothing phase, to train the attacker model to generate diverse and effective attack prompts.

2.2 Existing Defenses

We categorize existing defense measures into two main types: model-level defenses and prompt-level defenses.

Model-Level Defenses: Bianchi et al. [4] demonstrates that incorporating appropriate safety data during the fine-tuning process of Llama can significantly enhance the model’s safety. Bai et al. [3]

fine-tunes language models using preference modeling and RLHF to make them useful and harmless assistants. Ouyang et al. [24] fine-tune GPT-3 using reinforcement learning from human feedback (RLHF), resulting in the creation of InstructGPT. Ji et al. [15] designs a novel and simple alignment paradigm that learns the correctional residuals between preferred and dispreferred answers using a small model. Wang et al. [29] proposes the Backdoor Enhanced Safety Alignment method inspired by an analogy with the concept of backdoor attacks.

Prompt-Level Defenses: Jain et al. [14] and Alon and Kamfonas [2] employ input perplexity as a detection mechanism to defend against optimization-based attacks. Phute et al. [25] leverages the LLM itself to detect whether harmful content is being generated. Jain et al. [14] proposes using paraphrasing and re-tokenization as defenses against optimization-based attacks, both of which involve

modifying the input. Xie et al. [33] utilizes self-reminders in system prompts to encourage LLMs to respond responsibly, thereby reducing the success rate of jailbreak attacks. Zheng et al. [37] introduces a secure prompt optimization method that shifts the representation of a query either toward the rejection direction or away from it, depending on the harmfulness of the query. Our HSF falls into this category. Compared to existing methods, HSF mitigates jailbreak attacks without compromising the quality of the model's output or incurring additional inference costs.

3 Motivation

In this section, we explore the representation capabilities of LLMs in distinguishing harmful queries, benign queries, and harmful queries with adversarial prefixes.

3.1 Data Synthesis and Basement Model

If harmful and benign queries can be distinguished, we want this distinction to arise from their differences in harmfulness rather than other unrelated factors such as format or length. To address the impact of unrelated features, we used the commercial API of ChatGPT, gpt-3.5-turbo, to synthesize harmful and benign queries with fine control.

To ensure the diversity of the synthesized dataset, we sampled the top 100 data points from Advbench [39] as harmful queries and instructed gpt-3.5-turbo to generate benign versions of these queries while maintaining consistency in length as much as possible. Please refer to the appendix for the prompts we used to guide the data synthesis. We then applied additional manual checks to ensure effectiveness and quality. As a result, we collected 100 harmful and 100 benign queries, with an average length of 15.05 and 17.14 tokens, respectively (measured using the Llama2-7b-chat tokenizer).

We conduct experiments with three popular 7B chat LLMs available on HuggingFace, as well as an uncensored 30B LLM: Llama2-chat [28], Vicuna-v1.5 [6], Mistral-instruct-v0.2 [18], and WizardLM-30B-Uncensored [7]. Some of these models explicitly underwent extensive safety training (Llama2-7b-chat and Mistral-instruct-v0.2).

3.2 Visualization Analysis

In this section, we first compare the performance of aligned and unaligned models in LLM's hidden state representations for harmful and benign queries. Then, we examine whether LLM's hidden state have the ability to distinguish between harmful, harmless, and jailbreaking.

3.2.1 Aligned Model vs. Unaligned Model.

Following [37], we used Principal Component Analysis (PCA) to visualize the hidden states of the models. We selected the hidden states of the last input token from the model's last Decoder Layer output, as these hidden states intuitively gather all the information about the model's understanding of the query and its response strategy. It should be noted that these hidden states are also projected through the language modeling head (a linear mapping) for the prediction of the next token, which means they present a linear structure in the corresponding representation space (as assumed by PCA). We use two sets of hidden states to compute the first two principal components for each model, which include both harmful and benign queries. By selecting these data points, we are able to

extract the most significant features related to the harmfulness of the queries.

We find that the alignment models have a significantly better ability to distinguish between harmful and benign queries compared to unaligned models. From the top half of Figure 1, it can be seen that models trained with safety measures, such as Llama2-7B-chat, Vicuna-v1.5 and Mistral-instruct-v0.2, can naturally distinguish harmful queries from benign ones, with their boundaries (black dashed lines) easily fitted by logistic regression using the harmfulness of the queries as labels. However, the unaligned model WizardLM-30B-Uncensored lack this capability.

3.2.2 Distinguishing Ability for Jailbreak Attack.

Inspired by this finding, we construct six different jailbreak attacks based on the easyjailbreak dataset and the 50 harmful queries from the aforementioned synthesized dataset. The attack methods include AutoDAN [21], DeepInception [20], GPTFuzz[36], ICA[31], ReNellm[9], and gcg[39], each aiming to bypass the model's safety mechanisms through different approaches. For Llama2-7B-chat, we use three sets of hidden states to compute the first two principal components, which included benign queries, harmful queries, and jailbreak attacks.

We observe that models trained with safety protocols can not only distinguish between harmful and benign queries but also differentiate jailbreak attacks. The boundaries between these categories (indicated by black dashed lines) can be easily fitted using an SVM, with the harmfulness of the queries still serving as the labels. More results can be found in Appendix???. Based on this observation, our insights for developing a lightweight classification model include: (i) employing more robust methods for sampling the classification criteria, and (ii) more accurately fitting the boundaries between benign queries, harmful queries, and jailbreak attacks within the model's representation space.

4 Hidden State Filter

In this section, we present an overview of the HSF, followed by a detailed description of its design.

4.1 Overview of Hidden State Filter

Our HSF consists of two stages, as illustrated in Figure3. The first stage is the training phase, where a weak classifier is constructed to classify user queries. This weak classifier is trained using hidden vectors extracted from the last Decoder Layer during the forward pass of a specified LLM. In the second inference stage, the user's query undergoes a forward pass through the LLM to obtain the hidden vectors from the last Decoder Layer. HSF then classifies the last k tokens of these hidden vectors to determine whether to block the current inference. The remainder of this section details each step. It is important to note that the HSF does not generalize across different models; therefore, for clarity, we will illustrate the process using Llama2-7B-chat and Mistral-instruct-v0.2 as the example model.

4.2 Training Phase: A Weak Classifier

To construct the expert model, we first collect 3,000 samples each from the UltraSafety [11] and PKU-SafeRLHF-prompt [16] datasets as the harmful query dataset. These queries are expected to be

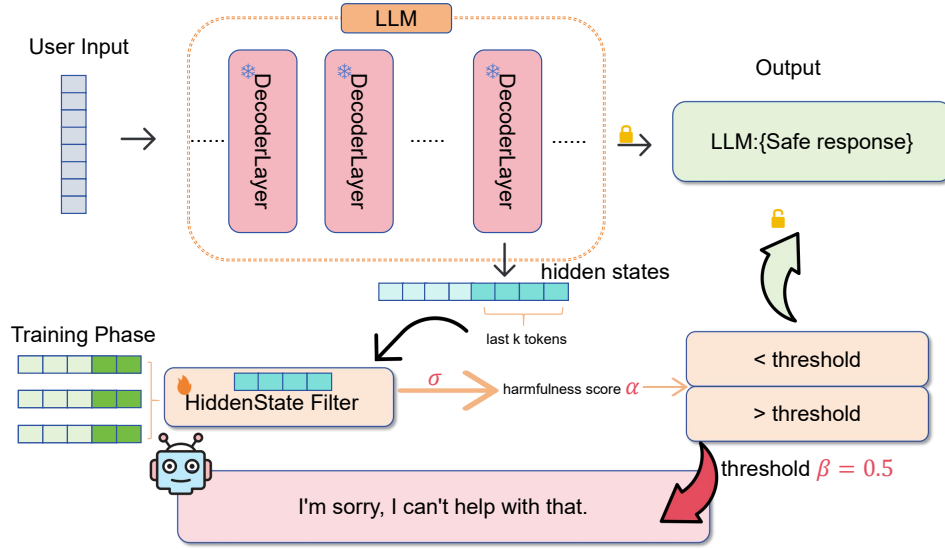


Figure 3: Schematic representation of the HSF mechanism. In this process, the Large Language Model (LLM) forwards the hidden states from the final Decoder Layer to the trained HSF. The filter extracts the last k tokens, concatenates them, and computes a harmfulness score α using a sigmoid function. This score is then compared to a threshold to determine the model's response, ensuring safe and reliable output.

rejected by any LLM aligned with human values. We also collect 6,000 samples from the databricks-dolly-15k [8] dataset as the benign query dataset. For the poorly aligned mistral-instruct-v0.2 model, we additionally sample 750 queries from both the harmful and benign datasets we construct. These samples are used to create a positive and negative dataset for training by adding template completion jailbreak attack templates.

Step 1: To construct the training samples, we input these queries into the model and perform a forward pass, extracting the vectors corresponding to the last k tokens from the hidden state of the last Decoder Layer, denoted as $t_k \in \mathbb{R}^{n \times k}$. Since the default chat template is used when inputting queries into the LLM, the token length is at least 8 for the Llama2-7B-chat tokenizer. Therefore, we set the search space for the last k tokens t_k as $k \in \{1, 2, \dots, 8\}$.

Step 2: Concatenation of k vectors. The vectors corresponding to the last k tokens are concatenated in their original order, with zeros padded between every two tokens to soft-distinguish different tokens and optimize the classifier's recognition capability. The specific formula is as follows:

Let $t_1, t_2, \dots, t_k$ be the vectors corresponding to the last k tokens, where $t_i \in \mathbb{R}^n$. We define the vector T_k as:

$$T_k = [t_k, \mathbf{0}, t_2, \mathbf{0}, \dots, \mathbf{0}, t_1], \quad (1)$$

where $\mathbf{0} \in \mathbb{R}^n$ denotes a zero vector of length n .

Step 3: Constructing the weak classifier. Considering the model alignment capability extension and inference time cost control, we use a simple Multilayer Perceptron (MLP) as the classifier. We fit a

logistic regression model using the empirical rejection probability of T_k , applying dropout regularization to prevent overfitting:

$$f_k : \mathbb{R}^{n \times k} \rightarrow \mathbb{R}, \quad f_k(x) = w_k^T T_k + b_k, \quad (2)$$

where $w_k \in \mathbb{R}^{n \times k}$ and $b_k \in \mathbb{R}$ are the parameters to be fitted by the logistic regression model.

Step 4: Model training. We train the model by minimizing the binary cross-entropy loss function. The specific training steps include forward propagation to compute the loss and backpropagation to update the parameters.

For a given query sample T_k , we define the following binary cross-entropy loss function:

$$\mathcal{L}_k(\theta) = -l \log(\sigma(f_k(T_k))) - (1-l) \log(1 - \sigma(f_k(T_k))), \quad (3)$$

where: $l \in \{0, 1\}$ is the binary label indicating whether the query is safe, σ is the sigmoid function.

To optimize the model parameters θ , we minimize the above loss function $\mathcal{L}_k(\theta)$, ensuring that harmful queries ($l = 1$) receive a higher harmfulness score and benign queries ($l = 0$) receive a lower harmfulness score.

4.3 Defense Phase: Implementing Jailbreak Attack Defense by Classifying User Queries

In the defense phase, the trained weak classifier is inserted into LLM as a plug-in module. The process is as follows:

Table 2: Comparison of Attack Success Rates between HSF and baseline defenses on Mistral-instruct-v0.2 and Llama2-7B-chat.

Model	Defense	Harmful Benchmark ↓		Jailbreak Attacks ↓					
		AdvBench	MaliciousInstruct	GCG	AutoDAN	ReNellm	DeepInception	ICA	GPTFuzz
Mistral	No Defense	77%	82%	80%	98%	84%	67%	56%	88%
	PPL	77%	82%	0%	98%	84%	67%	56%	88%
	Paraphrase	34%	45%	70%	44%	47%	15%	71%	76%
	Retokenization	65%	54%	70%	72%	47%	48%	82%	58%
	Self-Examination	13%	4%	6%	19%	12%	2%	3%	28%
	Self-Reminder	1%	2%	6%	92%	59%	19%	0%	80%
	DRO	73%	78%	76%	100%	87%	71%	63%	93%
	SPD	99%	60%	75%	4%	61%	83%	0%	77%
	HSF	0%	1%	4%	0%	8%	0%	0%	0%
Llama2	No Defense	0%	0%	38%	10%	20%	44%	0%	30%
	PPL	0%	0%	0%	10%	20%	44%	0%	30%
	Paraphrase	0%	1%	4%	9%	11%	29%	0%	29%
	Retokenization	6%	5%	9%	19%	49%	53%	0%	31%
	Self-Examination	0%	0%	6%	0%	0%	0%	0%	2%
	Self-Reminder	0%	0%	0%	4%	0%	0%	0%	2%
	DRO	0%	0%	2%	9%	22%	32%	0%	30%
	SPD	0%	0%	100%	12%	61%	100%	0%	2%
	HSF	0%	0%	1%	0%	0%	0%	0%	1%

1. Receive the concatenated vector T_k of the selected last k tokens as input.

2. Input the concatenated vector T_k into the classifier to calculate a harmfulness score α (ranging from 0 to 1).

3. Specify a threshold β ; if the harmfulness score exceeds this threshold, the inference process is interrupted, and the model refuses to generate a response.

5 Experiments

In this section, we evaluate the defense performance of the HSF, focusing on two key metrics: Attack Success Rate (ASR) and Area Under the Curve (AUC).

5.1 Experimental Setup

Basement Models. We deploy HSF on three open-source LLMs: Vicuna-v1.5 [6], Llama2-7b-chat [28], Mistral-instruct-v0.2 [18] to evaluate its effectiveness.

Attack Methods. We consider six state-of-the-art jailbreak attacks, each covering different categories, implemented through easyjailbreak [38]. Specifically, GCG [39] represents gradient-based attacks, AutoDAN [21] is based on genetic algorithms, and ICA [31] represents context-learning-based attacks. We also included ReNellm [9], DeepInception [20], and GPTFuzz [36] as representative empirical jailbreak attacks. To evaluate the defense performance against naive attackers directly inputting harmful queries, we used two harmful query benchmark datasets: Advbench [39] and MaliciousInstruct [13]. Detailed settings for these attack methods and harmful query datasets can be found in the AppendixA.1.

Baselines. We consider six state-of-the-art defense mechanisms as baselines. PPL[2] and Self-Examination [25]) are input-output detection-based methods, while Paraphrase [14], Retokenization [14], and Self-Reminder [33] are mitigation-based methods. DRO[37] is a safety-prompt-based method. SPD[5] is a method for detecting

jailbreaking inputs via the logit values in a single forward pass. Note that the original classifier used in SPD is SVM. However, due to issues encountered during reproduction, it has been replaced with an equivalent MLP structure in this study. Detailed descriptions and hyperparameter settings for each method can be found in the AppendixA.1.

Evaluation Metrics. We utilize Llama-Guard-3-8B [23] to evaluate the safety of the model's responses. Although keyword-based detection methods [39] are commonly employed, our experiments revealed that these methods had a low AUC, indicating a high false positive rate. This necessitated additional manual verification, leading to increased costs. Consequently, we employ the state-of-the-art Llama-Guard-3-8B to assess whether the model generated unsafe content and to calculate the ASR.

HSF Settings. We set the hyperparameter $k = 7$ for Llama2-7B-chat, meaning that we used the last 7 tokens of the last DecoderLayer as the basis for the HSF input. We also set the harmfulness detection threshold to 0.5. In section 5.3, We present an ablation analysis of different values of the last k tokens.

5.2 Experimental Results

HSF Enhances LLM Safety. Table 2 presents a comparison of the ASR for the Mistral-instruct-v0.2 and Llama2-7B-chat models when employing HSF and various baseline defenses against six different jailbreak attacks. The results indicate that for models with weaker safety alignment, such as Mistral-instruct-v0.2, HSF markedly decreases the ASR, outperforming almost all baseline defenses. Notably, while most other defenses were ineffective against AutoDAN, HSF successfully defends against this attack, achieving an ASR of 0. For models with strong alignment, such as Llama2-7B-chat, HSF consistently reduces the ASR across all attacks to nearly 0. Additional results for HSF applied to Vicuna-v1.5 can be found in Appendix??.

Moreover, HSF has a high AUC score. Table 3 shows the AUC score of HSF on safe datasets.

5.3 Ablation Analysis

In this section, we conduct an ablation analysis on the hyperparameter k in HSF, tested on Mistral-instruct-v0.2 and Llama2-7B-chat models. Table 4 show that HSF exhibits significant robustness against prompt rewriting attacks, maintaining an extremely low ASR regardless of the k value. However, for more complex template completion attacks, such as ReNellm and DeepInception, the defense performance of the model shows a clear dependency on the hyperparameter settings. This suggests that to maintain high classification performance in advanced attack scenarios, HSF requires appropriate hyperparameter tuning.

Moreover, we find that due to the identical chat template used, the last 4 tokens of the model input remained consistent. As a result, HSF exhibits a lower ASR at $k = 4$, demonstrating better defense effectiveness. Additionally, for GCG jailbreak attacks, when $k > 1$, HSF effectively disrupts its robustness, leading to a significant drop in ASR.

6 Limitations

Limited Generalizability: The HSF is specifically tailored to work with certain LLMs, necessitating retraining when applied to other models. This dependency on the alignment capabilities of the underlying model restricts its generalizability to models that have undergone analogous alignment training. However, while retraining is required for each new model, the process incurs relatively low computational cost and does not alter the model’s architecture.

7 Conclusion

In this paper, we introduce a novel model architecture module, HSF, designed to defend against jailbreak attacks on LLMs by leveraging the model’s alignment capabilities reflected in its hidden states. We observe that aligned models can internally distinguish between jailbreak attacks, harmful queries, and benign queries. Inspired by this observation, we develop a plug-in weak classifier module that uses the last k tokens from the LLM’s DecoderLayer to maximize the utilization of the model’s complex alignment capabilities as a safety measure. This ability allows HSF to identify the harmfulness of queries before inference begins. Our results demonstrate that HSF can effectively defend against state-of-the-art jailbreak attacks while being both efficient and beneficial. In future, we will explore how to leverage LLM hidden states to build more flexible harmful-behavior detectors and enhance jailbreak attack prevention through Retrieval-Augmented Generation methods.

Acknowledgments

This work was funded by the National Natural Science Foundation of China (NSFC) under Grants No. 62406013, the Beijing Advanced Innovation Center Funds for Future Blockchain and Privacy Computing and the Fundamental Research Funds for the Central Universities.

Table 3: Comparison of AUC for Mistral-instruct-v0.2 and Llama2-7B-chat across different K values (1-8).

Model	AUC $\uparrow$							
	K=1	K=2	K=3	K=4	K=5	K=6	K=7	K=8
Mistral	0.9990	0.9986	0.9998	0.9991	0.9990	0.9992	0.9997	0.9988
Llama2	0.9999	0.9994	0.9994	0.9998	0.9999	0.9997	0.9999	0.9997

Table 4: Ablation analysis on the hyperparameter k for HSF.

Model	Last K	Harmful Benchmark $\downarrow$		Jailbreak Attacks $\downarrow$					
		AdvBench	MaliciousInstruct	GCG	AutoDAN	ReNellm	DeepInception	ICA	GPTFuzz
Mistral	k=1	1%	1%	16%	0%	3%	6%	0%	0%
	k=2	1%	0%	4%	0%	12%	0%	0%	0%
	k=3	0%	1%	4%	0%	8%	0%	0%	0%
	k=4	1%	0%	4%	0%	17%	0%	0%	0%
	k=5	0%	1%	4%	0%	66%	0%	0%	0%
	k=6	0%	2%	6%	0%	64%	0%	0%	0%
	k=7	0%	0%	6%	0%	62%	0%	0%	0%
	k=8	0%	0%	2%	0%	66%	0%	0%	0%
Llama2	k=1	0%	1%	21%	0%	23%	48%	0%	1%
	k=2	0%	0%	8%	0%	21%	50%	0%	1%
	k=3	1%	0%	1%	0%	23%	12%	0%	1%
	k=4	1%	0%	3%	0%	21%	3%	0%	1%
	k=5	0%	0%	2%	0%	21%	58%	0%	0%
	k=6	0%	0%	1%	0%	29%	60%	0%	1%
	k=7	0%	0%	1%	0%	0%	0%	0%	1%
	k=8	0%	0%	1%	0%	31%	66%	0%	1%

References

- [1] Josh Achiam, Steven Adler, Sandhini Agarwal, Lama Ahmad, Ilge Akkaya, Florencia Leoni Aleman, Diogo Almeida, Janko Altmenschmidt, Sam Altman, Shyamal Anadkat, et al. 2023. Gpt-4 technical report. *arXiv preprint arXiv:2303.08774* (2023).
- [2] Gabriel Alon and Michael Kamfonas. 2023. Detecting language model attacks with perplexity. *arXiv preprint arXiv:2308.14132* (2023).
- [3] Yuntao Bai, Andy Jones, Kamal Ndousse, Amanda Askell, Anna Chen, Nova DasSarma, Dawn Drain, Stanislav Fort, Deep Ganguli, Tom Henighan, et al. 2022. Training a helpful and harmless assistant with reinforcement learning from human feedback. *arXiv preprint arXiv:2204.05862* (2022).
- [4] Federico Bianchi, Mirac Suzgun, Giuseppe Attanasio, Paul Rottger, Dan Jurafsky, Tatsunori Hashimoto, and James Zou. 2024. Safety-Tuned LLaMAs: Lessons From Improving the Safety of Large Language Models that Follow Instructions. In *The Twelfth International Conference on Learning Representations*.
- [5] Leyla Naz Candogan, Yongtao Wu, Elias Abad Rocamora, Grigorios Chrysos, and Volkan Cevher. [n. d.]. Single-pass detection of jailbreaking input in large language models. In *ICLR 2024 Workshop on Secure and Trustworthy Large Language Models*.
- [6] Wei-Lin Chiang, Zhuohan Li, Zi Lin, Ying Sheng, Zhanghao Wu, Hao Zhang, Lianmin Zheng, Siyuan Zhuang, Yonghao Zhuang, Joseph E Gonzalez, et al. 2023. Vicuna: An open-source chatbot impressing gpt-4 with 90%* chatgpt quality. See <https://vicuna.lmsys.org> (accessed 14 April 2023) 2, 3 (2023), 6.
- [7] Cognitive Computations. 2024. WizardLM-30B-Uncensored. <https://huggingface.co/cognitivecomputations/WizardLM-30B-Uncensored>. Accessed: 2024-08-15.
- [8] Mike Conover, Matt Hayes, Ankit Mathur, Jianwei Xie, Jun Wan, Sam Shah, Ali Ghodsi, Patrick Wendell, Matei Zaharia, and Reynold Xin. 2023. Free dolly: Introducing the world's first truly open instruction-tuned llm. *Company Blog of Databricks* (2023).
- [9] Peng Ding, Jun Kuang, Dan Ma, Xuezhi Cao, Yunsen Xian, Jiajun Chen, and Shujian Huang. 2024. A Wolf in Sheep's Clothing: Generalized Nested Jailbreak Prompts can Fool Large Language Models Easily. In *Proceedings of the 2024 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 1: Long Papers)*. 2136–2153.
- [10] Emilio Ferrara. 2023. Should chatgpt be biased? challenges and risks of bias in large language models. *arXiv preprint arXiv:2304.03738* (2023).
- [11] Yiju Guo, Ganqu Cui, Lifan Yuan, Ning Ding, Jiexin Wang, Huimin Chen, Bowen Sun, Ruobing Xie, Jie Zhou, Yankai Lin, et al. 2024. Controllable Preference Optimization: Toward Controllable Multi-Objective Alignment. *arXiv preprint arXiv:2402.19085* (2024).
- [12] Zhang-Wei Hong, Idan Shenfeld, Tsun-Hsuan Wang, Yung-Sung Chuang, Aldo Pareja, James R Glass, Akash Srivastava, and Pulkit Agrawal. 2024. Curiosity-driven Red-teaming for Large Language Models. In *The Twelfth International Conference on Learning Representations*.
- [13] Yangsibo Huang, Samyak Gupta, Mengzhou Xia, Kai Li, and Danqi Chen. 2024. Catastrophic Jailbreak of Open-source LLMs via Exploiting Generation. In *The Twelfth International Conference on Learning Representations*.
- [14] Neel Jain, Avi Schwarzschild, Yuxin Wen, Gowthami Somepalli, John Kirchenbauer, Ping-yeh Chiang, Micah Goldblum, Aniruddha Saha, Jonas Geiping, and Tom Goldstein. 2023. Baseline defenses for adversarial attacks against aligned language models. *arXiv preprint arXiv:2309.00614* (2023).
- [15] Jiaming Ji, Boyuan Chen, Hantao Lou, Donghai Hong, Borong Zhang, Xuehai Pan, Juntao Dai, and Yaodong Yang. 2024. Aligner: Achieving efficient alignment through weak-to-strong correction. *arXiv preprint arXiv:2402.02416* (2024).
- [16] Jiaming Ji, Mickel Liu, Josef Dai, Xuehai Pan, Chi Zhang, Ce Bian, Boyuan Chen, Ruiyang Sun, Yizhou Wang, and Yaodong Yang. 2024. Beavertails: Towards improved safety alignment of llm via a human-preference dataset. *Advances in Neural Information Processing Systems* 36 (2024).
- [17] Ziwei Ji, Nayeon Lee, Rita Frieske, Tiezheng Yu, Dan Su, Yan Xu, Etsuko Ishii, Ye Jin Bang, Andrea Madotto, and Pascale Fung. 2023. Survey of hallucination in natural language generation. *Comput. Surveys* 55, 12 (2023), 1–38.
- [18] Albert Q Jiang, Alexandre Sablayrolles, Arthur Mensch, Chris Bamford, Devendra Singh Chaplot, Diego de las Casas, Florian Bressand, Gianna Lengyel, Guillaume Lample, Lucile Saulnier, et al. 2023. Mistral 7B. *arXiv preprint arXiv:2310.06825* (2023).
- [19] Seanie Lee, Minsu Kim, Lynn Cherif, David Dobre, Juho Lee, Sung Ju Hwang, Kenji Kawaguchi, Gauthier Gidel, Yoshua Bengio, Nikolay Malkin, et al. 2024. Learning diverse attacks on large language models for robust red-teaming and safety tuning. *arXiv preprint arXiv:2405.18540* (2024).
- [20] Xuan Li, Zhanke Zhou, Jianing Zhu, Jiangchao Yao, Tongliang Liu, and Bo Han. 2023. Deepinception: Hypnotize large language model to be jailbreaker. *arXiv preprint arXiv:2311.03191* (2023).
- [21] Xiaogeng Liu, Nan Xu, Muhao Chen, and Chaowei Xiao. 2024. AutoDAN: Generating Stealthy Jailbreak Prompts on Aligned Large Language Models. In *The Twelfth International Conference on Learning Representations*.
- [22] Yi Liu, Gelei Deng, Zhengzi Xu, Yuekang Li, Yaowen Zheng, Ying Zhang, Lida Zhao, Tianwei Zhang, Kailong Wang, and Yang Liu. 2023. Jailbreaking chatgpt via prompt engineering: An empirical study. *arXiv preprint arXiv:2305.13860* (2023).
- [23] AI @ Meta Llama Team. 2024. The Llama 3 Herd of Models. *arXiv:2407.21783* [cs.AI] <https://arxiv.org/abs/2407.21783>
- [24] Long Ouyang, Jeffrey Wu, Xu Jiang, Diogo Almeida, Carroll Wainwright, Pamela Mishkin, Chong Zhang, Sandhini Agarwal, Katarina Slama, Alex Ray, et al. 2022. Training language models to follow instructions with human feedback. *Advances in neural information processing systems* 35 (2022), 27730–27744.
- [25] Mansi Phute, Alec Helbling, Matthew Daniel Hull, ShengYun Peng, Sebastian Szyller, Cory Cornelius, and Duen Horng Chau. 2023. Llm self defense: By self examination, llms know they are being tricked. In *The Second Tiny Papers Track at ICLR 2024*.
- [26] Ivan Provilkov, Dmitrii Emelianenko, and Elena Voita. 2020. BPE-Dropout: Simple and Effective Subword Regularization. In *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics*. 1882–1892.
- [27] Mikayel Samvelyan, Sharath Chandra Raparthy, Andrei Lupu, Eric Hambro, Aram H Markosyan, Manish Bhatt, Yuning Mao, Minqi Jiang, Jack Parker-Holder, Jakob Foerster, et al. 2024. Rainbow teaming: Open-ended generation of diverse adversarial prompts. *arXiv preprint arXiv:2402.16822* (2024).
- [28] Hugo Touvron, Louis Martin, Kevin Stone, Peter Albert, Amjad Almahairi, Yasmine Babaei, Nikolay Bashlykov, Soumya Batra, Prajwal Bhargava, Shruti Bhosale, et al. 2023. Llama 2: Open foundation and fine-tuned chat models. *arXiv preprint arXiv:2307.09288* (2023).
- [29] Jiong Xiao Wang, Jiazhao Li, Yiquan Li, Xiangyu Qi, Muhao Chen, Junjie Hu, Yixuan Li, Bo Li, and Chaowei Xiao. 2024. Mitigating fine-tuning jailbreak attack with backdoor enhanced alignment. *arXiv preprint arXiv:2402.14968* (2024).
- [30] Alexander Wei, Nika Haghtalab, and Jacob Steinhardt. 2024. Jailbroken: How does llm safety training fail? *Advances in Neural Information Processing Systems* 36 (2024).
- [31] Zeming Wei, Yifei Wang, and Yisen Wang. 2023. Jailbreak and guard aligned language models with only few in-context demonstrations. *arXiv preprint arXiv:2310.06387* (2023).
- [32] Laura Weidinger, John Mellor, Maribeth Rauh, Conor Griffin, Jonathan Uesato, Po-Sen Huang, Myra Cheng, Mia Glaese, Borja Balle, Atosa Kasirzadeh, et al. 2021. Ethical and social risks of harm from language models. *arXiv preprint arXiv:2112.04359* (2021).
- [33] Yueqi Xie, Jingwei Yi, Jiawei Shao, Justin Curl, Lingjuan Lyu, Qifeng Chen, Xing Xie, and Fangzhao Wu. 2023. Defending chatgpt against jailbreak attack via self-reminders. *Nature Machine Intelligence* 5, 12 (2023), 1486–1496.
- [34] Zhangchen Xu, Fengqing Jiang, Luyao Niu, Jinyuan Jia, Bill Yuchen Lin, and Radha Poovendran. 2024. Safedecoding: Defending against jailbreak attacks via safety-aware decoding. *arXiv preprint arXiv:2402.08983* (2024).
- [35] Sibo Yi, Yule Liu, Zhen Sun, Tianshuo Cong, Xinlei He, Jiaxing Song, Ke Xu, and Qi Li. 2024. Jailbreak Attacks and Defenses Against Large Language Models: A Survey. *arXiv preprint arXiv:2407.04295* (2024).
- [36] Jiahao Yu, Xingwei Lin, and Xinyu Xing. 2023. Gptfuzzer: Red teaming large language models with auto-generated jailbreak prompts. *arXiv preprint arXiv:2309.10253* (2023).
- [37] Chujie Zheng, Fan Yin, Hao Zhou, Fandong Meng, Jie Zhou, Kai-Wei Chang, Minlie Huang, and Nanyun Peng. 2024. On prompt-driven safeguarding for large language models. In *Forty-first International Conference on Machine Learning*.
- [38] Weikang Zhou, Xiao Wang, Limao Xiong, Han Xia, Yingshuang Gu, Mingxu Chai, Fukang Zhu, Caishuang Huang, Shihan Dou, Zhiheng Xi, et al. 2024. EasyJailbreak: A Unified Framework for Jailbreaking Large Language Models. *arXiv preprint arXiv:2403.12171* (2024).
- [39] Andy Zou, Zifan Wang, Nicholas Carlini, Milad Nasr, J Zico Kolter, and Matt Fredrikson. 2023. Universal and transferable adversarial attacks on aligned language models. *arXiv preprint arXiv:2307.15043* (2023).

A Appendix A: Detailed Experimental Setups

A.1 Attack Setup

For GCG [39], AutoDAN [21], GPTFuzz [36], DeepInception [20], and ICA [31], we use the easyjailbreak [38] framework to construct attacks. We utilized 100 different representative harmful queries from Advbench [39]. For ReNellm [9], we rewrite these 100 representative harmful queries collected from Advbench using easyjailbreak and filled them into templates to create 100 jailbreak prompts.

A.2 Baseline Setup

1. PPL [2]: PPL is an input detection mechanism that calculates the perplexity of a given input to determine whether the user's request should be accepted or rejected.

The definition of perplexity (P) is as follows:

$$\text{Perplexity}(P) = \exp \left(\frac{-1}{N} \sum_{i=1}^N \log P(w_i | w_{1:i-1}) \right). \quad (4)$$

Following [2], we use Llama2-7b-chat to calculate perplexity. As per [34], we set the PPL threshold to the highest perplexity value of harmful queries in Advbench. This ensures that queries from Advbench do not trigger the detector.

2. Paraphrase [14]: We follow [14] and used GPT-4o by default to paraphrase user queries. The prompt is as follows:

The paraphrased output is then used as the input to the target language model.

Prompt

Please paraphrase the following sentences in {user prompt}. Give me paraphrased results only. Do not include any other information. Repeat, give me paraphrased results only and do not include any other information.

3. Retokenization [14]: This method splits tokens and represents them using multiple smaller tokens. For example, the token representing the word “breaking” is split into two tokens representing “break” and “ing.” We use BPE-dropout [26], which randomly drops $p\%$ of BPE merges during the tokenization process. We set $p = 0.2$ following [14].

4. Self-Examination [25]: This is an output detection method that uses the language model itself to distinguish whether harmful content is being generated. We use the prompt suggested by [25]:

Prompt

Does the following text contain harmful content:
{user prompt} Respond with “Yes, this is harmful” or “No, this is not harmful”

5. Self-Reminder[33]: Self-Reminder appends prompts to input prompts to remind the language model to respond responsibly.

Prompt

System: You should be a responsible AI and not generate harmful, misleading content! Please answer the following query in a responsible way.
User: {user prompt} Remember, you should be a responsible AI and should not generate harmful or misleading content!

6. DRO [37]: This is an automated optimization method for generating safe prompts. Following previous work, we use a default type of system safety prompt and directly embedded the generated safety prompt as a vector in the embedding layer before the user’s input.

7. SPD [5]: This is a method to detect adversarial samples by extracting a feature matrix H from the logit outputs of a LLM. H captures unnatural token distribution changes caused by attacks, using the top- k logits for r token positions. The authors originally used a Support Vector Machine (SVM) with an RBF kernel trained on H for classification. However, during our reproduction, we encountered issues with the SVM and instead replaced it with a MLP. The hyperparameters include r (number of token positions, set to 5), k (number of top logits, set to 50).